ISL-ViT-Tiny: A Compact Factorised Vision Transformer for Isolated-Word Indian Sign Language Recognition, Evaluated on Held-Out Recording Sessions

Yalamanchi Kushal[1], Nethi Kushala Kumar[1], Dhruseek Rishi Menon[1]

[1]Amrita School of Computing, Amrita Vishwa Vidyapeetham, Coimbatore, India.

Abstract

Isolated-word recognition of Indian Sign Language (ISL) on a wearable device needs a small model and an accuracy figure measured on recording conditions the model has not seen. We present ISL-ViT-Tiny, a factorised spatio-temporal Vision Transformer that reads three detected crops per frame (left hand, right hand, face) over 16 frames, adds 21 hand-joint landmarks per hand to the hand tokens, and is trained on INCLUDE (262 words) after masked-image pretraining on iSign. Evaluation holds out whole INCLUDE recording sessions, identified from camera capture counters. On the 472 held-out clips of the 50 deployed words, with predictions restricted to those words and six-view test-time augmentation, the released 4-bit model (2,069,700 bytes; 3,894,214 parameters) reaches 85.9% mean top-1 accuracy over three seeds and 96.4% top-5. Hand landmarks add 12.8 points over a pixel-only control trained with the same recipe, and quantisation-aware training keeps the full-precision accuracy within seed noise. The same recipe scores 98.5% on a random-video split and 70.1% on held-out sessions, which quantifies how far conventional splits overstate accuracy on this corpus. A learned super-resolution front end raises 180p accuracy from 61.0% to 76.0%, and the integrated application reproduces the offline evaluation clip for clip.

Keywords: Indian Sign Language recognition, Vision Transformer, spatio-temporal modelling, hand landmarks, 4-bit quantisation, evaluation protocol, super-resolution GAN

1 Introduction

Indian Sign Language is the primary language of a large Deaf community in India; the INCLUDE authors cite more than five million users [1]. An automatic recogniser for isolated ISL words could support communication with non-signers, and a camera on a pair of smart glasses is one place such a recogniser could run. That target sets two requirements that pull against each other. The network must fit a memory and compute budget far below that of standard video models, and its reported accuracy must describe what happens when the signer, room and lighting differ from the training recordings.

The second requirement is the harder one to meet with public ISL data. INCLUDE, the main labelled word-level corpus, contains 4,287 videos of 263 words recorded by experienced signers [1]; the copy used here has 4,257 usable clips of 262 words. A word has a median of 15 clips, many of them back-to-back takes of the same signer in the same session. A random split of such data puts near-identical takes on both sides of the train/test boundary, so a high score can reflect recognition of the recording rather than of the sign. The problem is not specific to sign language: Kapoor and Narayanan document train–test leakage across machine-learning-based science [2]. Within sign language, AUTSL reports 95.95% for its baselines on random splits and 62.02% on its user-independent split [3]. Published INCLUDE results of 85.6% (263 words) and 94.5% (INCLUDE-50) [1], and 93.5 to 94.7% for pose-based models [4], were obtained on the dataset's own split; Section 5.3 measures how much such splits inflate accuracy on this corpus.

Transformer video models [5, 6] and pose-based sign recognisers [4, 7, 8] give strong results on sign data, and recent work targets edge hardware [9, 10]. In the sources we reviewed (Section 2) we did not find an ISL word recogniser that reports, in one place, a hard size budget, a measured quantisation cost, a split that holds out whole recording sessions, and an application tested against the offline evaluation.

The project's objectives were to (i) reach at least 75% top-1 accuracy on 50 deployable words measured on held-out recording sessions; (ii) keep the model near 2 MB; (iii) use an evaluation protocol that does not reward memorising a session; (iv) integrate the model into a working interface; and (v) test a generative adversarial network (GAN) in a position where its contribution can be isolated. This report describes the implementation at commit `3330b87` of the public repository and the measurements made on it.

The contributions are:

1. A task-specific compact recogniser: a factorised spatial-then-temporal Transformer over hand and face crops, with crop position, detection status and normalised hand-joint features added to the crop tokens (Section 3.5), sized and quantised to a 2 MB budget.
2. Controlled measurements on one fixed test set: the landmark stream (+12.8 points, paired $p \leq 5 \times 10^{-9}$ in each seed), post-training versus quantisation-aware 4-bit compression, temporal length, crop resolution and training duration (Section 5).
3. A recording-session-aware evaluation built from capture identifiers, with a matched measurement of how much a random split inflates accuracy (about 28 points for one recipe) (Section 3.10).
4. An integrated application whose prediction route reproduces the offline per-clip outcome for three classifier variants and three low-resolution front ends, on 50 held-out clips each (Section 5.8).
5. A controlled super-resolution study that separates the benefit of a learned upscaler from the benefit of the adversarial loss (Section 5.7).

2 Related work

ISL datasets and recognisers.

INCLUDE [1] is the reference labelled corpus for isolated ISL words; its best model, a pretrained feature extractor and encoder with a trained decoder, reached 85.6% on 263 words and 94.5% on the 50-word subset. CISLR [11] has a vocabulary of about 4,700 words with roughly 1.5 videos per word, so its authors frame recognition as a one-shot task; their prototype baseline reached 16.81% top-1 on CISLR v1.0-a. iSign [12] provides a large collection of ISL video–sentence pairs and benchmark tasks. Recent ISL systems include a CNN–ViT conversational agent reporting 97.5% accuracy under optimised conditions on a domain-specific dataset [13] and a two-handed dynamic ISL recogniser [14]; Damdoo and Kumar survey the field [15].

Video and pose models for sign recognition.

The Vision Transformer [16] and the data-efficient DeiT training recipe [17] made patch-token models practical on modest datasets. For video, ViViT [5] studies variants that factorise spatial and temporal attention, and TimeSformer [6] compares divided space–time attention with joint attention. Sign Language Transformers [18] applied Transformers to continuous recognition and translation. Pose-based recognisers operate on body and hand keypoints: SPOTER [7] normalises hand poses in a local coordinate frame and reports 63.18% on WLASL100; SAM-SLR [8] fuses skeleton, RGB and depth streams; SignBERT [19] pretrains a hand-model-aware representation; OpenHands [4] releases pose-based models for six sign languages and reports 93.5% (SL-GCN) on INCLUDE, rising to 94.7% with self-supervised pretraining. Our model combines the two families: pixels of detected regions, plus hand landmarks from MediaPipe [20, 21] added to the visual tokens rather than processed by a separate graph network.

Compact models and quantisation.

Integer quantisation with simulated rounding during training [22] and the straight-through gradient estimator [23] are the standard basis of quantisation-aware training; Nagel et al. review post-training and quantisation-aware methods [24]. Q-BERT [25] and GPTQ [26] give each block of weights its own scale (group-wise scaling). For sign recognition at the edge, TinyMSLR [9] reports fewer than 2.7 M parameters and 7.2 MB on a 20-class isolated-sign benchmark built from two corpora with a signer-independent protocol, and a Kannada Sign Language study [10] deploys quantised LSTM and Transformer models on a smartphone.

Evaluation protocols.

WLASL [27] collects more than 2,000 words from over 100 signers, and AUTSL [3] publishes a user-independent benchmark split and reports the random-versus-user-independent gap quoted in Section 1. INCLUDE ships without signer identifiers; the protocol in Section 3.10 reconstructs recording sessions from the camera's running clip counter.

Super-resolution for recognition.

SRGAN [28], ESRGAN [29] and Real-ESRGAN [30] sharpen upscaled images by training the generator against a discriminator [31]. Blau and Michaeli show that perceptual quality and distortion measures such as PSNR trade off against each other [32]. Whether super-resolution helps a downstream task is an empirical question: Dai et al. examined it for several vision tasks [33], and Haris et al. trained super-resolution with a detection loss [34]. Our study asks the narrower question of what the adversarial term adds to learned upscaling for a frozen sign recogniser.

Table 1 places the closest works beside this project. Numbers are as reported by each paper; datasets, vocabularies and splits differ, so the column is not a ranking.

3 Methodology

3.1 Task definition

Each input is a short video containing one isolated sign. The model f_θ outputs logits over the $K = 262$ INCLUDE words it was trained on. The deployed product answers from a fixed subset $\mathcal{D}$ of $|\mathcal{D}| = 50$ words, declared before any run from the word list of the deployed split file (mask 50.py:45). The deployed prediction is

$$\hat{y} = \arg\max_{k \in \mathcal{D}} \ \bar{p}_k, \qquad \bar{\mathbf{p}} = \frac{1}{|\mathcal{V}|} \sum_{v \in \mathcal{V}} \operatorname{softmax}\big(f_\theta(x_v)\big), \quad (1)$$

where x_v is view v of the clip and $\mathcal{V}$ is the set of test-time views (Section 3.11). Restricting the argmax to $\mathcal{D}$ is what a 50-word product does with this network; Section 5.1 also reports the unrestricted 262-word accuracy.

3.2 Data and vocabulary

The classifier is trained only on INCLUDE [1] (CC-BY-4.0): 4,257 clips of 262 words at 1920×1080 pixels and 25

Table 1 Closest prior work. Results are as reported by each source.

Work	Data and task	Model and input	Protocol and reported result	Relation to this project
INCLUDE [1]	ISL, 263 and 50 words	pretrained extractor and encoder, trained decoder; RGB video	dataset split; 85.6% (263), 94.5% (50)	same corpus; we hold out sessions
Open-Hands [4]	6 sign languages incl. INCLUDE	LSTM, BERT, ST-GCN, SL-GCN; pose keypoints	INCLUDE: 93.5% (SL-GCN), 94.7% with pretraining	keypoints only; dataset split
CISLR [11]	ISL, about 4,700 words	prototype one-shot; I3D features	16.81% top-1 (v1.0-a)	our cross-corpus test set
SPOTER [7]	ASL WLASL, Argentinian LSA64	Transformer; 2D pose	WLASL100 63.18%; LSA64 100%	normalised hand pose, as in our features
SAM-SLR [8]	Turkish AUTSL	GCN and CNN ensemble; skeleton, RGB, depth	signer-independent challenge; 98.42% (RGB track)	multimodal fusion; much larger
AUTSL [3]	Turkish, 226 signs	CNN and LSTM; RGB-D	95.95% random vs 62.02% user-independent	split changes the conclusion
Tiny-MSLR [9]	20 classes from PHOENIX-2014T and CSL	ConvNeXt-Tiny and Swin, distilled; RGB	signer-independent; 99.01% validation; under 2.7 M parameters, 7.2 MB	edge target; smaller task
Kannada SLR [10]	Kannada SL, 33 words	LSTM, BiLSTM, Transformer; MediaPipe landmarks	94.19 to 95.71%; 1,098 KB on phone	phone deployment
Nedungadi et al. [13]	ISL, domain dataset	CNN and ViT; RGB	97.5% under optimised conditions	ISL application; protocol differs
This work	ISL INCLUDE, 262 trained, 50 deployed	factorised ViT, 4-bit; 3 crops and hand landmarks	held-out sessions; 85.9% masked-50, 3 seeds; 2,069,700 B	–

frames per second, all from the original 1080p sources. Two other corpora enter as unlabelled or test data. iSign [12] supplied 17,896 clips (one per source video, after a hand-detection filter) for self-supervised pretraining, and CISLR v1.5-a [11] supplied 609 clips of 218 shared words as a cross-corpus test.

3.3 Region extraction and missing detections

One code path builds the training cache, the test cache and the application input (crops.py:490, reused by predict.py:42). Thirty-two frames are sampled uniformly from the clip, resized to a height of 720 pixels and letterboxed to 1280×720 (crops.py:163). This resize is also where the super-resolution front end is inserted (Section 3.9). MediaPipe Holistic [20] locates the hands and face; a hand box is a square around the 21 hand landmarks enlarged 1.5 times. When Holistic misses a hand, a region around the forearm predicted by the body pose is upscaled to 256 pixels and passed to a dedicated hand landmarker [21] (crops.py:305). If both fail, the nearest real box is carried into the gap and the frame is flagged as interpolated (crops.py:362). Crops are stored at 128 pixels and resized to 64 pixels at model input.

Two observations during development shaped this design. A fallback box synthesised from pose geometry alone can land on clothing or background, so every crop carries a provenance code (direct, region rescue, interpolated, missing) that the model receives as input. MediaPipe's detectors also keep state between videos even in image mode, so the detectors are rebuilt at the start of every video (crops.py:413); the 472 test clips were extracted this way into a separate cache, and every headline number uses that cache.

3.4 Landmark features

For each hand h and frame t, MediaPipe returns joints $\mathbf{J}_{t,h} \in \mathbb{R}^{21\times 3}$ in frame-normalised coordinates. The model computes its own invariant features (isl_vit.py:312), so every inference path computes them identically. Let $\tilde{\mathbf{J}} = \mathbf{J}\,\mathrm{diag}(a,1,1)$ with $a = 16/9$, which undoes the letterbox aspect ratio, and let $\tilde{\mathbf{j}}_0$ be the wrist row. The handshape descriptor is

$$\mathbf{s}_{t,h} = \frac{\mathrm{vec}\left(\tilde{\mathbf{J}}_{t,h} - \mathbf{1}\,\tilde{\mathbf{j}}_{t,h,0}^{\top}\right)}{\max\left(\max_i \|(\tilde{\mathbf{j}}_{t,h,i} - \tilde{\mathbf{j}}_{t,h,0})_{xy}\|_2,\ 10^{-3}\right)} \in \mathbb{R}^{63}, \quad (2)$$

which removes hand position and distance from the camera. Sign location is the wrist relative to the shoulder midpoint $\mathbf{c}_t$, divided by the shoulder width w_t: $\mathbf{u}_{t,h} =$

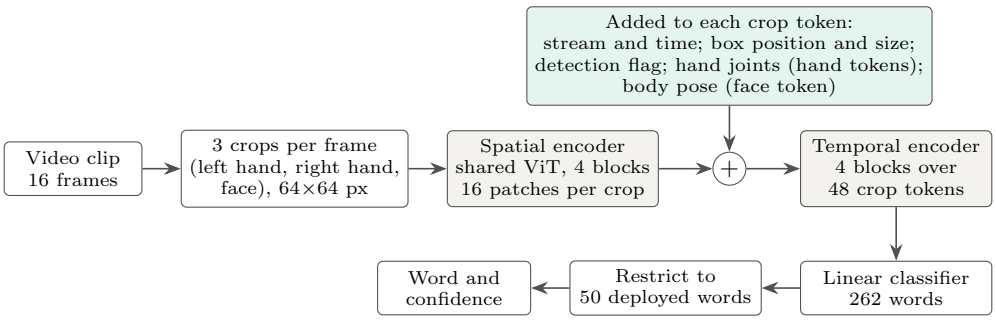


Fig. 1 ISL-ViT-Tiny. Attention is factorised: first within each crop, then across the 48 crop tokens of the clip. Shaded boxes are Transformer stacks; the green box holds the additive token-conditioning terms of Eq. (3).

Source: isl_vit.py:136-299 (model), isl_vit.py:312-344 (landmark features), mask50.py:56-108 (50-word restriction). Parameter counts from report/final/scripts/measure_model.py.

$(\tilde{\mathbf{j}}_{t,h,0} - \mathbf{c}_t)_{xy}/w_t \in \mathbb{R}^2$. Seven upper-body pose points expressed in the same body frame, with their visibilities, form a 21-dimensional pose vector $\mathbf{r}_t$. If either shoulder has visibility of 0.3 or less, every body-relative quantity is set to zero. The 65-dimensional hand vector $[\mathbf{s}; \mathbf{u}]$ and the pose vector each pass through a two-layer perceptron (Linear–GELU–Linear, 192 units) whose last layer starts at zero, so at the first training step the landmark model computes exactly what the pixel-only model computes.

3.5 Recognition model

Figure 1 shows the network (isl_vit.py:136). The input is $T = 16$ frames $\times$ $S = 3$ streams of 64×64 RGB crops.

Spatial encoder. One encoder with shared weights processes all $T \cdot S = 48$ crops (isl_vit.py:87). A 16×16 patch embedding gives 16 patch tokens plus a class token, which pass through four pre-norm Transformer blocks of width 192 with three heads; the output class token is the crop embedding $\mathbf{e}_{t,s} \in \mathbb{R}^{192}$. Width and head count match DeiT-Tiny [17], so ImageNet and self-supervised weights load without projection.

Token conditioning. Cropping discards where a hand is, yet location distinguishes many signs. Each crop token therefore receives additive terms:

$$\begin{aligned}\mathbf{z}_{t,s} = \mathbf{e}_{t,s} &+ \mathbf{E}^{\text{str}}_s + \mathbf{E}^{\text{time}}_t + g(\mathbf{b}_{t,s}) + (1 - d_{t,s})\,\mathbf{m} \\ &+ \mathbb{I}[s \in \{\text{L}, \text{R}\}]\, q_{t,s}\, \phi_{\text{hand}}([\mathbf{s}; \mathbf{u}]_{t,s}) \\ &+ \mathbb{I}[s = \text{F}]\, o_t\, \phi_{\text{pose}}(\mathbf{r}_t),\end{aligned} \tag{3}$$

with learned stream and time embeddings $\mathbf{E}$; box geometry $\mathbf{b} = (x_c, y_c, \text{size})$ mapped by a zero-initialised perceptron g; detection indicator d and a learned missing-detection embedding $\mathbf{m}$; hand-present indicator q; pose-valid indicator o; and landmark projections ϕ (Section 3.4). L, R and F denote the left-hand, right-hand and face streams. The landmark terms add 91,008 parameters and no tokens.

Temporal encoder. The 48 conditioned tokens and a class token pass through four more Transformer blocks, and a linear layer maps the final class token to 262 logits. Attention therefore runs over 17 tokens inside each crop and 49 tokens across the clip, instead of the $48 \times 17 = 816$ tokens that joint space–time attention would process. This is the factorised-encoder arrangement studied in ViViT [5].

3.6 Training procedure

The released model was trained on the 2,845 training clips of the session-disjoint split with validation folded in (Section 3.10), for a fixed 1,000 epochs, keeping the final weights (train.py:244; launch scripts run_landmarks.sh and run_lm_s2_resume.sh). The optimiser is AdamW [35] with a peak learning rate of 5×10^{-4}, 10 warm-up epochs, cosine decay to 1%, batch 64 and weight decay 0.05 on weight matrices only; the spatial encoder learns at 0.1 times the base rate. Regularisation uses label smoothing of 0.1 [36], mixup with $\alpha = 0.2$ on half the batches [37], stochastic depth up to 0.1 [38], gradient clipping at 1.0 and an exponential moving average of the weights. Augmentation applies temporal jitter within uniform segments, scale-jittered crop windows, horizontal flips that swap the two hand streams and their joints, brightness, contrast and per-channel gain changes, greyscale, stream dropout, and a per-clip resolution jitter. Seeds 0, 1 and 2 give three independent models. The effective recipe of each run is stored in its `summary.json`.

Final weights are kept rather than a best-validation checkpoint, because validation clips come from the training sessions; on the 262-word split this choice raised test top-1 from 22.1% to 25.3% (Table A1, row E6).

3.7 Self-supervised initialisation

Both encoders are initialised from masked-image pretraining on iSign in the style of SimMIM [39] (pretrain.py:182; launch run_isign_pipeline.sh). A learned mask token replaces 75% of the patch tokens, sampled jointly over time, stream and space; a two-block decoder predicts per-patch-normalised pixels, and the loss is the mean squared error on masked patches. Interpolated crops are excluded from the loss so that the model does not learn to reconstruct stale background. Position and time embeddings are resized when the pretrained geometry differs from the fine-tuning geometry (train.py:75).

3.8 Four-bit quantisation

Every weight matrix (tensors with at least two dimensions) is split into groups G of 128 consecutive weights and

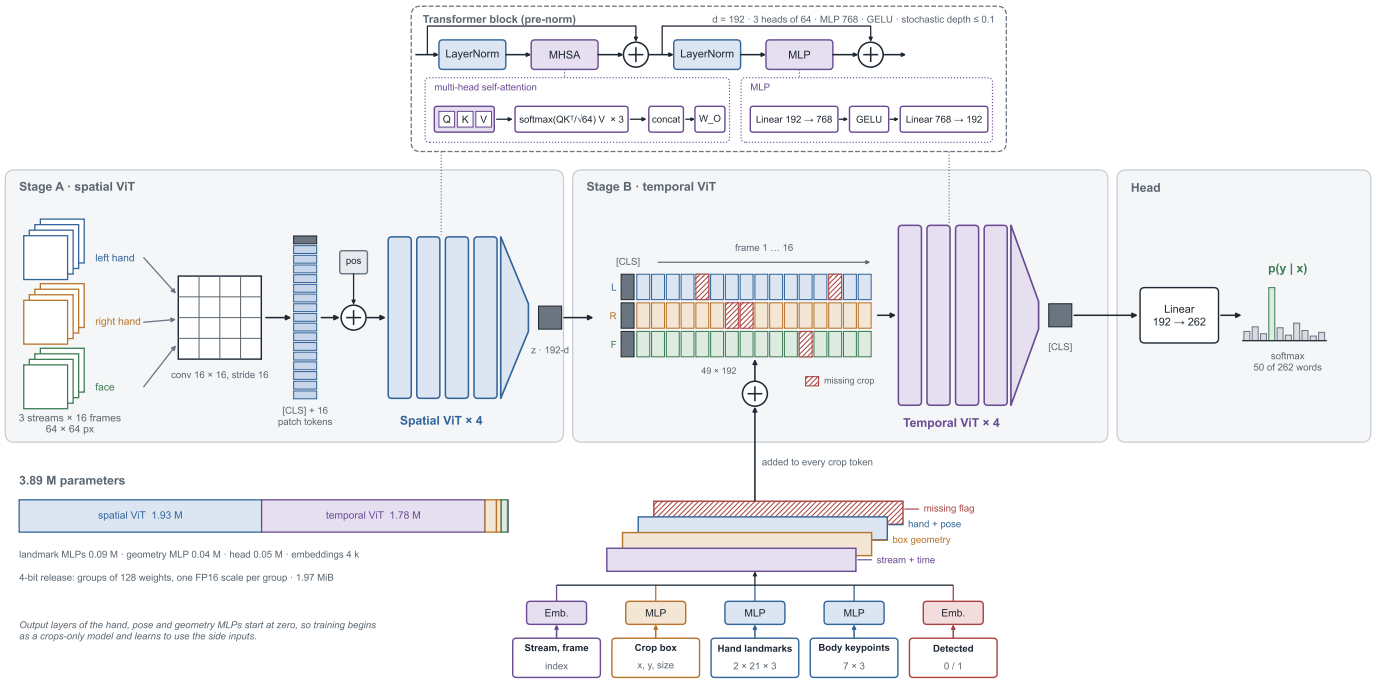


Fig. 2 Network architecture of ISL-ViT-Tiny (3,894,214 parameters), read left to right. *Stage A*, a spatial Vision Transformer, encodes one crop at a time: the 48 crops of a clip (left hand, right hand and face, 64 × 64 pixels, 16 frames) share its weights. A 16 × 16 convolution with stride 16 cuts each crop into 16 patch tokens of width 192; a learned [CLS] token (dark cell) is prepended, a position embedding is added, and four transformer blocks follow. The funnel marks the readout of the [CLS] output as the crop embedding $z \in \mathbb{R}^{192}$. The dashed inset at the top opens one pre-norm block, used unchanged in both stages: LayerNorm, three-head self-attention, residual sum, LayerNorm, a 192 → 768 → 192 MLP with GELU, residual sum. *Stage B* arranges the 48 embeddings as three streams (rows L, R, F) by 16 frames; hatched cells are crops whose detector did not fire. The offset sheets below the grid are tensors of the same shape that are summed into the tokens before Stage B. Stream and time embeddings and the crop-box MLP apply to every token, the hand-landmark MLP to the two hand rows, the body-keypoint MLP to the face row, and a learned missing-detection vector to the hatched cells. The last layer of each of these MLPs starts at zero, so training begins from a crops-only model. A second [CLS] token makes 49 tokens; after four blocks its output passes through a linear layer to 262 word logits, and at inference the softmax is restricted to the 50 deployed words. The bar at lower left splits the parameters by module.

Drawn by report/architecture/make_architecture_diagrams.py; full description in docs/ARCHITECTURE_DIAGRAMS.md.

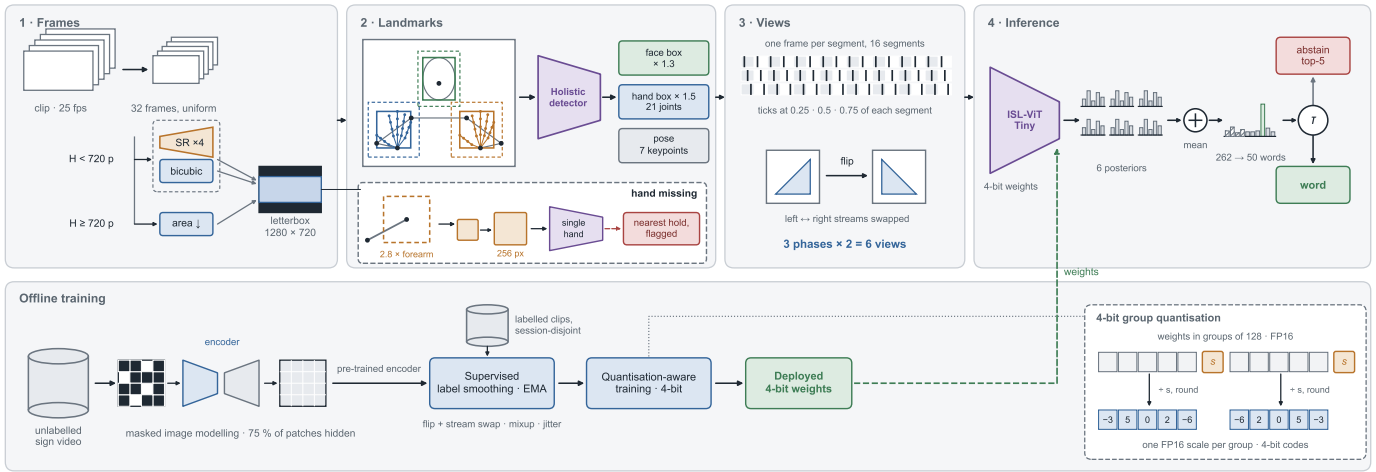


Fig. 3 End-to-end pipeline. The four upper panels run for every clip; the band below runs once, offline. Funnels are learned models, offset squares are stacks of frames and small bar charts are probability vectors. (1) *Frames*: 32 frames are sampled uniformly. Frames shorter than 720 pixels are enlarged by bicubic interpolation or, as an option, by the ×4 super-resolution generator; taller frames are area-downsampled; all are letterboxed to 1280 × 720. (2) *Landmarks*: a holistic detector, rebuilt for every clip, returns pose, face and hand landmarks, and the face and hand boxes are squares around them enlarged 1.3 and 1.5 times. The dashed box is the path for a missing hand: a region of 2.8 forearm lengths around the wrist is upsampled to 256 pixels and passed to a single-hand detector, and if that also fails the nearest real box is reused and flagged as interpolated. That flag is the detection indicator of Fig. 2. (3) *Views*: one frame is taken from each of 16 equal segments at three phase offsets (ticks), and each selection is also mirrored with the hand streams swapped, giving six views. (4) *Inference*: the 4-bit classifier gives one posterior over 262 words per view; the six are averaged, restricted to the 50 deployed words and renormalised, and the threshold τ decides between emitting the top word and abstaining with the top five. *Offline training*: masked image modelling on unlabelled sign video, with 75% of patches hidden, pretrains the encoder; supervised training on the session-disjoint split and 4-bit quantisation-aware training follow, and the dashed green arrow carries the resulting weights into panel 4. The inset shows how they are stored: groups of 128 weights, each with one FP16 scale, rounded to 4-bit codes.

Drawn by report/architecture/make_architecture_diagrams.py; full description in docs/ARCHITECTURE_DIAGRAMS.md.

quantised symmetrically (export.py:223):

$$\sigma_G = \mathrm{fp16}\Big(\frac{\max_{w\in G}|w|}{7}\Big), \qquad (4)$$
$$c_w = \mathrm{clamp}\,\big(\mathrm{round}(w/\sigma_G), -8, 7\big), \qquad \hat{w} = \sigma_G\, c_w.$$

Two 4-bit codes are packed per byte with one FP16 scale per group; embeddings, normalisation parameters and biases stay in FP16 (export.py:202, export.py:246). For post-training quantisation (PTQ), Eq. (4) is applied once to the trained FP32 weights (int4_eval.py). Quantisation-aware training (QAT) continues training for a fixed 300 epochs at a learning rate of 5×10^{-5}, computing the forward pass with $\hat{w}$ and applying the gradients to the full-precision weights, a straight-through estimator [22, 23] implemented by swapping tensors around each step (qat.py:50; launch run_lm_qat.sh). QAT and the exporter call the same target-selection and rounding functions, so the model trained is the model saved. The release files are written by release.py and read by export.py:181, which rebuilds FP32 weights from the stored codes at load time.

3.9 Super-resolution front end and GAN study

For video below 720 pixels in height, the frame reader enlarges frames before detection. The default is bicubic interpolation; the alternative is a ×4 generator applied at the same point (crops.py:163). The generator is a compact VGG-style network of the kind distributed with the Real-ESRGAN code base, trained on Real-ESRGAN-style synthetic degradations [30]: 3×3 convolutions at low resolution, a final pixel shuffle, and a residual over nearest-neighbour upsampling (sr/models.py:32). Training pairs come from high-resolution hand, face and frame patches of training sessions only, degraded by random blur, resizing, noise and JPEG compression (sr/degrade.py:122). Figure 4 shows the comparison design. The arms are:

- **SR-L1 (no GAN)**: a 621,424-parameter generator trained for 100,000 iterations with pixel L1 loss only.
- **SR-GAN v1**: SR-L1 fine-tuned for 60,000 iterations with L1, VGG19 perceptual [40, 41] and adversarial [31] losses against a spectrally normalised U-Net discriminator [42, 43] (sr/models.py:51).
- **SR-GAN v2**: v1 trained for 80,000 further iterations with a fresh discriminator.
- **SR-GAN v3 and its control**: a 2,039,568-parameter generator with a relativistic average adversarial loss [44], H.264 degradation, and a loss on hand landmarks predicted by a differentiable ResNet-18 [45] heatmap network with soft-argmax readout [46] (sr/train.py:100). The control is the same generator and hand loss without the discriminator.

Test inputs are the 472 test clips degraded deterministically inside the frame reader by area downscaling and one JPEG encode: 360p (quality 60), 240p (50) and 180p (40) (sr/degrade.py:35; launch run_gan_study.sh, run_gan_study_v2.sh, run_gan_v3_test.sh). The three released classifiers are scored unchanged on each front end's output. Before the first test run, the rule for a verdict was fixed: "helps" requires a higher mean, all three seeds in the same direction, and pooled exact McNemar $p < 0.05$ (gan_report.py:98).

3.10 Evaluation protocol

INCLUDE file names carry the camera's running clip counter (`MVI_xxxx`). Clips of one word whose counters differ by at most 5 form a *take-group* of back-to-back recordings (splits.py:118); the gap histogram is bimodal, and thresholds from 5 to 20 give the same partition. Clustering the counters of the whole corpus with the same gap gives 13 contiguous *session blocks* (splits.py:170), the recording-session structure that INCLUDE's metadata makes available. The session-disjoint split assigns whole blocks to test by random search over 20,000 candidate assignments, scored mainly on how many test words keep at least six training clips (splits.py:209). Blocks 0, 5 and 6 form the test side. Figure 5 contrasts the three protocols.

For final models, the validation clips (taken from training sessions at take-group granularity) are folded back into training, giving 2,845 training clips over 262 words. The held-out blocks contain 1,412 clips: 1,010 form the test set, and 402 belong to words with fewer than six training clips and are marked unused. The deployed test set is the 472 test clips of the 50 deployed words (_test472_paths.csv); the audit script confirmed that this list equals the test rows of the deployed split and that every headline result file scores exactly these clips.

3.11 Metrics, test-time views and statistics

Top-1 and top-5 accuracy are the share of clips whose true word is ranked first or within the first five; balanced accuracy is the mean per-word recall (tta.py:65). *Masked-50* applies Eq. (1) with $\mathcal{D}$ the deployed words; *wide-262* takes the argmax over all 262 outputs. Test-time augmentation (TTA) averages softmax probabilities over six views: three temporal phases (0.25, 0.5 and 0.75 of each sampling segment), each with and without a horizontal flip that swaps the hand streams (tta.py:46). Each headline figure is the mean of three seeds. Paired comparisons on identical clips use the exact two-sided McNemar test [47] on the discordant clips, per seed and pooled over seeds. Five seeds of one earlier configuration gave a top-1 standard deviation of 1.9 points on 1,010 clips (runs/sd263_ssl_foldval*); differences below about 5 points between single runs are therefore read as seed noise.

3.12 Application path

The application is a standard-library HTTP server (serve.py:102) and a seven-screen browser interface built into one page from islvit/ui/src. `POST /predict` receives raw video bytes and runs the same extraction, landmark and view functions as the offline evaluation (serve.py:50). The demo launcher (demo.py:69) replaces the server's classifier with one that applies the 50-word restriction exactly as in Eq. (1), and can bind a super-resolution generator into the

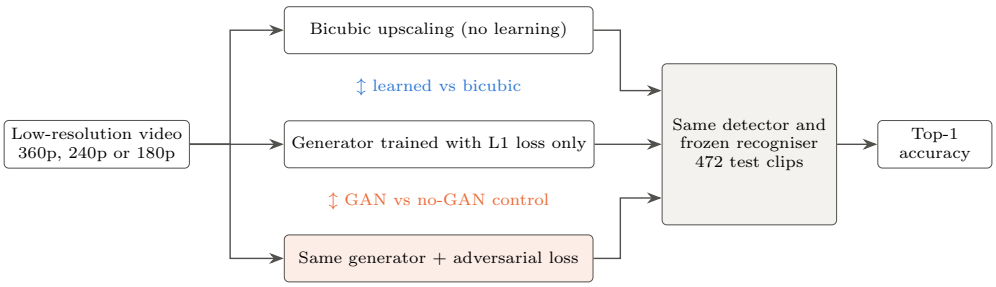


Fig. 4 Design of the super-resolution comparison. Only the upscaling step changes between arms: bicubic versus a learned upscaler measures what learning to upscale buys, and a GAN versus the same generator without its discriminator measures what the adversarial term buys. v3 has its own matched control.

Source: sr/models.py (networks), sr/train.py:137 (training), crops.py:163 (insertion point); drivers run_gan_study.sh, run_gan_v3.sh. Generator SHA-256 hashes are recorded in runs/gan_study/detection/*.json.

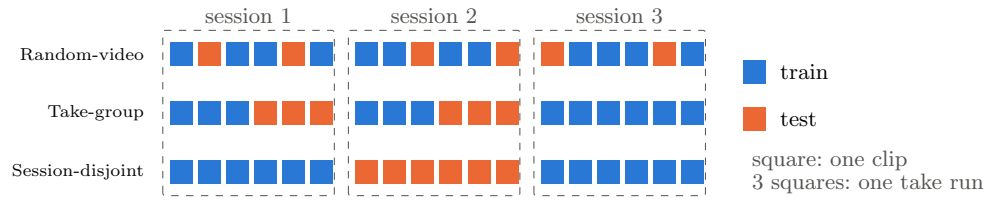


Fig. 5 Three split protocols on the same recordings (schematic). A session contains several words, and the signer records each word as a run of near-identical takes. Random-video splits put takes of one run on both sides; take-group splits keep a run together but let one session supply both training and test clips; session-disjoint splits hold out whole sessions.

Measured on the split files with blocks rebuilt from file names (report/final/scripts/audit_results.py): the 262-word random-video split has 666 take-groups on both sides of the train/test boundary, the take-group split has all 13 blocks on both sides, and the training split of the final models (full263_session-disjoint-foldval.csv) and the deployed split (vocab50clean_session-disjoint.csv) have none.

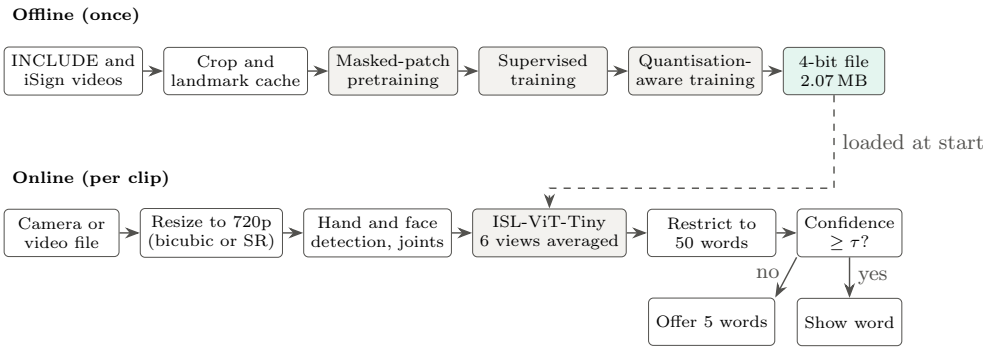


Fig. 6 End-to-end system: offline training and compression, and the per-clip inference path shared by the evaluation and the application. Shaded boxes are learned stages.

Source: online path predict.py:42-107, serve.py:50-99, demo.py:69-118; training commands run_isign_pipeline.sh, run_landmarks.sh, run_lm_qat.sh; release file release.py.

frame reader. The model is chosen with `--run` at launch. A prediction whose confidence is below a threshold τ (default 0.40) goes to a screen listing five candidates, and a user's correction is appended to a log. Figure 6 shows the offline and online paths.

3.13 Software and hardware

Experiments ran on one desktop: NVIDIA GeForce RTX 3060 (12 GB), AMD Ryzen 5 5500, 32 GB RAM, Windows 11, Python 3.11.15, PyTorch 2.6.0 with CUDA 12.4, MediaPipe 1.0.0 (Tasks API), timm 1.0.28 and OpenCV. Training seeds were 0, 1 and 2; split construction used seed 1337.

4 Experimental setup

Table 2 lists the settings of the principal experiments. Results in Sections 5.1 to 5.5 use the same 472 test clips unless the table says otherwise. The launch script and result files for every experiment are listed in Appendix A.

5 Results

5.1 Main result

The released 4-bit landmark model reached 85.9% top-1 (seeds 84.5, 87.5 and 85.8), 96.4% top-5 and 85.8% balanced accuracy, measured as masked-50 with six-view TTA on the 472 held-out-session test clips and averaged

Table 2 Settings of the principal experiments.

Experiment	Models compared	Test set	Seeds	Metric
Main result and variants	pixel FP32; landmark FP32; landmark PTQ INT4; landmark QAT INT4 (released)	472 clips, 50 words, held-out sessions	3 each	masked-50, 6-view TTA
Protocol comparison	one 50-word pixel recipe on random-video and session-disjoint splits	200 and 472 clips (each protocol's own test set)	1 and 2	top-1, single view
Temporal length	8, 16, 24, 32 frames from one 32-frame cache, 50-word pixel recipe	472 clips	1 (16 frames: 2)	top-1, single view and TTA
Crop resolution	64, 112, 128 pixel input, early 8-frame recipe	INCLUDE-50 take-group, 159 clips	1	top-1
Cross-corpus	released models on CISLR	609 CISLR clips	3	top-1
Super-resolution	bicubic, SR-L1, SR-GAN v1, v2, v3, v3 control; frozen INT4 classifiers	472 clips degraded to 360p, 240p, 180p	3	masked-50, TTA; McNemar
Application route	INT4 landmark, FP32 landmark, FP32 pixel (seed 0); INT4 with bicubic, SR-L1 and SR-GAN v2 at 180p; all through `POST /predict`	50 of the 472 clips (one per word)	1	agreement with offline outcome

Table 3 Controlled variants on the 472 held-out clips (masked-50, 6-view TTA, mean of three seeds). Each row differs from the one above in a single factor.

Variant	Precision	Parameters	Stored size	Top-1	Top-5	Balanced
Pixels only	FP32	3,803,206	15.2 MB[a]	73.7	93.5	73.3
Pixels + landmarks	FP32	3,894,214	15.6 MB[a]	86.5	96.5	86.5
Landmarks, PTQ	INT4, group 128	3,894,214	as QAT	83.4	95.8	83.3
Landmarks, QAT (released)	INT4, group 128	3,894,214	2,069,700 B	85.9	96.4	85.8

Per-seed top-1: pixels only 75.0/73.7/72.2; landmarks 87.3/86.9/85.4; PTQ 83.1/85.2/82.0; QAT 84.5/87.5/85.8. [a]Parameters × 4 bytes. 2,069,700 B = 1.974 MiB = 2.07 MB.

over three independently trained seeds. Answering from all 262 words gives 72.0% top-1 (69.7, 75.8, 70.3) and 92.7% top-5 on the same clips; a single centre view with the mask gives 85.5%; on the full 262-word test set (1,010 clips, single view) the same models reach 64.7% top-1. The values are read from runs/f16_262w_lm_s*_qat4/masked50_clean.json (keys `masked_tta`, `wide_tta`, `masked_plain`) and each run's `summary.json`; the evaluator is mask50.py:56 and the split vocab50clean_session-disjoint.csv. During this audit the three files in release/ were re-scored on the local test cache, and the per-clip outcomes matched the tracked files exactly for all three seeds (report/final/audit/rescore_release_s*.json).

5.2 Model variants

The variants compared here share one architecture and differ in input streams, numerical precision, temporal length or preprocessing, so each comparison isolates one change. Table 3 and Fig. 7 compare the four variants that share the test set, the training data and the recipe.

Landmark stream.

Adding the landmark stream, with nothing else changed, raised top-1 from 73.7% to 86.5%. The gain replicated in every seed: 76, 83 and 88 clips gained against 18, 21 and 26 lost ($p = 1.2 \times 10^{-9}$, 6.7×10^{-10} and 4.6×10^{-9}). The pipeline already ran MediaPipe to place crop boxes; the landmark model uses joints that the pixel-only model discarded.

Quantisation.

Rounding the FP32 landmark weights to 4 bits after training gave 83.4%; quantisation-aware training reached 85.9%. Pooled over seeds, QAT gained 77 clips and lost 41 relative to PTQ ($p = 0.0012$). Against FP32, QAT gained 37 clips and lost 45 (pooled $p = 0.44$): the released 4-bit model matches its FP32 parent within seed noise.

Compute and latency.

Table 4 reports measurements made for this paper. Model-only CPU latency was 20.9 ms per view. The end-to-end server route, which decodes a 1080p clip and runs MediaPipe over 32 frames, took a median of 4.3 s per clip on the same CPU, so detection and decoding, not the Transformer, dominate the end-to-end time.

5.3 Effect of the split protocol

The matched 50-word comparison gave 98.5% on the random-video split (one seed, 200 clips) and 70.1% on held-out sessions (69.9% and 70.3%, two seeds, 472 clips), a gap of 28.4 points. With the early 8-frame recipe on 262 words, accuracy fell from 94.5% (random-video) to 29.3% (take-group) and 22.1% (session-disjoint). The random-video score is mostly recognition of the recording: the

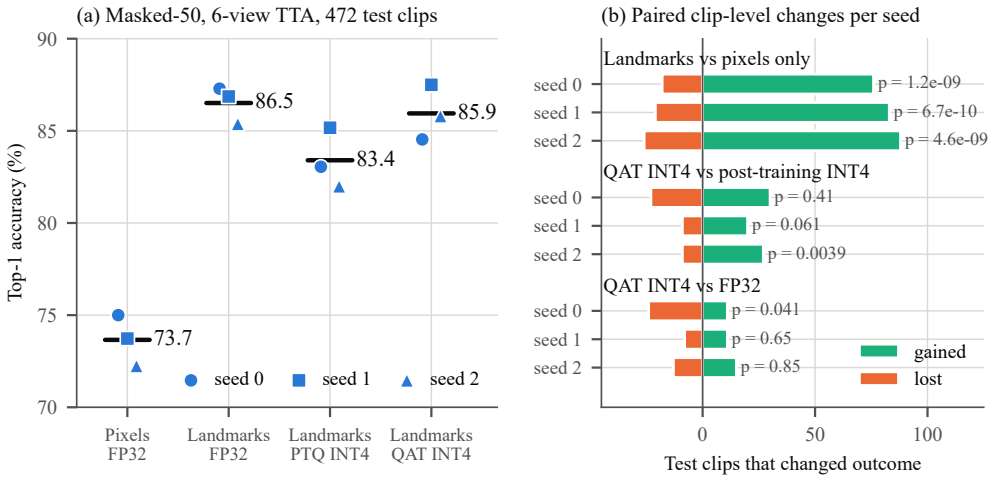


Fig. 7 Controlled variant comparison on the deployed test set. (a) Top-1 accuracy of each seed (markers) and the three-seed mean (bar). (b) For each seed, the test clips that the second model gets right and the first gets wrong (gained) or the reverse (lost), with the exact McNemar p for that seed.

Data: runs/f16_262w_s*/masked50_clean.json, runs/f16_262w_lm_s*/masked50_clean.json, runs/f16_262w_lm_s*/ptq4_clean.json, runs/f16_262w_lm_s*_qat4/masked50_clean.json (keys `masked_tta.top1`, `hits_masked_tta`); training run_landmarks.sh, run_lm_qat.sh; statistics and plot report/final/scripts/audit_results.py, make_figures.py.

Table 4 Compute and runtime of the released model, measured on the desktop described in Section 3.

Quantity	Value	Timing boundary and method
Parameters (landmark model, 262 outputs)	3,894,214	counted on the instantiated network; spatial encoder 1,930,944, temporal blocks 1,779,456, landmark projections 91,008
Multiply–accumulates per view	1.65 G (16 frames); 0.82 G (8 frames)	`torch.utils.flop_counter`; matrix products and convolutions only
Model forward pass, one view, CPU	20.9 ms median (IQR 20.2–21.9 ms)	released seed-0 file, batch 1, FP32 arithmetic, 6 threads, 100 runs after warm-up, idle machine
Server route, one clip, CPU	4.3 s median (10 clips)	`POST /predict` on a 1080p source: decoding, detection and landmarks, 6 views, 50-word mask
Server route, one clip, GPU	3.7 s median (50 clips)	as above, model and detector on the RTX 3060

Sources: report/final/data/model_measurements.json (measure_model.py); report/final/data/app_integration_int4_lm_s0.json and app_integration_int4_lm_s0_cpu.json (app_integration_test.py).

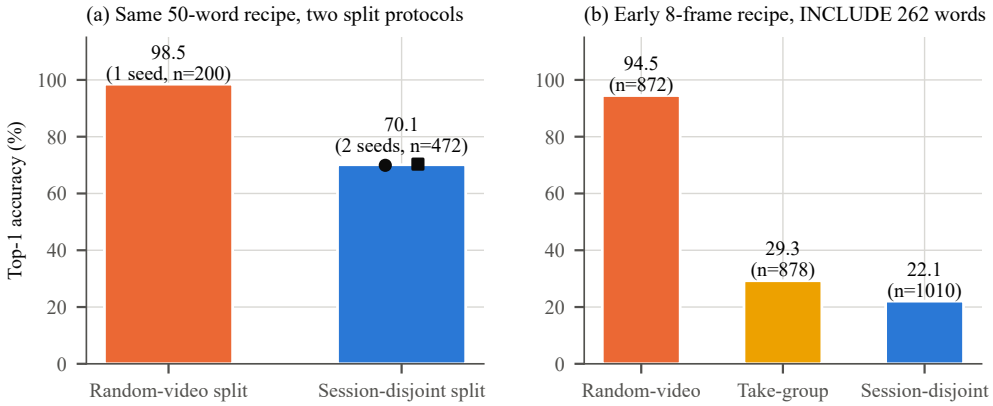


Fig. 8 Accuracy of one recipe under different split protocols. (a) A 50-word pixel-only recipe (16 frames, 2,000 epochs, iSign initialisation) trained on a random-video split and on the session-disjoint split. (b) An early 8-frame, 262-word recipe trained on three split files. Labels give each protocol's test-set size n.

Data: (a) runs/baseline_random_s0, runs/f16_clean_s0, runs/f16_clean_s1 (key `test.top1`); splits vocab50_random-video.csv, vocab50clean_session-disjoint.csv. (b) runs/full263_v2_random-video, runs/full263_v2_take-group, runs/sd_full263; launch run_phase2.sh.

take-group step removes near-duplicate takes and the session step removes shared recording conditions.

5.4 Temporal length, resolution and training budget

Going from 8 to 16 frames raised single-view top-1 from 60.0% to 70.1% (two-seed mean); 24 and 32 frames gave

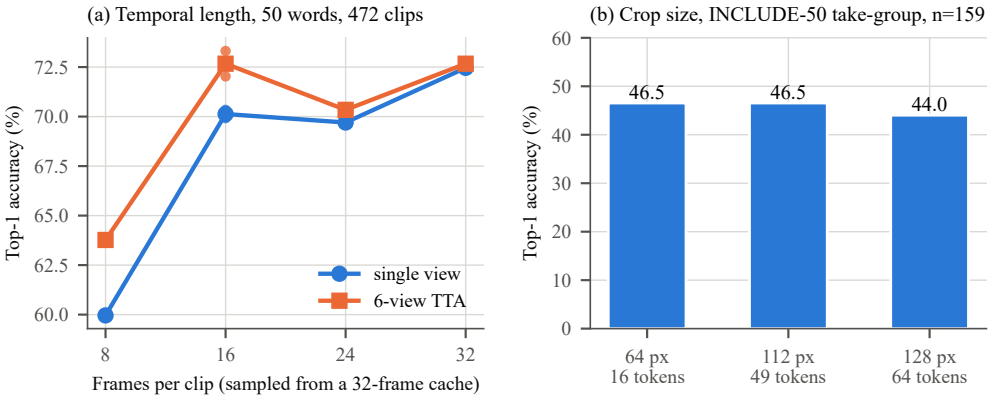


Fig. 9 Temporal length and crop resolution. (a) Top-1 of the 50-word pixel-only recipe with 8, 16, 24 or 32 frames sampled from the same 32-frame cache, single view and six-view TTA; at 16 frames the line is the mean of two seeds (dots). (b) Top-1 at three crop sizes, early 8-frame recipe, INCLUDE-50 take-group split (159 clips, one seed).

Data: (a) runs/f8_clean_s0, f16_clean_s0, f16_clean_s1, f24_clean_s0, f32_clean_s0 (keys `test`, `test_tta`); launch run_frames32.sh. (b) runs/abl64__take-group, abl112__take-group, abl128__take-group; launch run_ablation_res.sh.

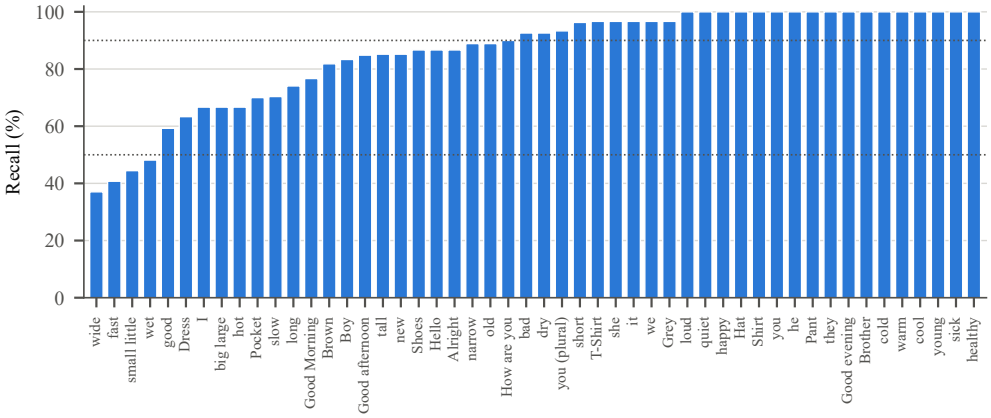


Fig. 10 Per-word recall of the released model on the 472 test clips, pooled over the three released seeds (about 9 to 11 test clips per word per seed), sorted. Dotted lines mark 50% and 90%.

Data: `hits_masked_tta` and `video_paths` in runs/f16_262w_lm_s*_qat4/masked50_clean.json; computed by make_figures.py.

69.7% and 72.5% from one seed each, within seed noise of the 16-frame result, so the model uses 16 frames at half the inference cost of 32. Crop size is set at 64 pixels: 64, 112 and 128 pixels gave 46.5%, 46.5% and 44.0% at 0.82, 2.34 and 3.03 GMACs per view (measured with measur e_model.py). On the 50-word tier, with a training split that also contained 76 CISLR clips and the same 472 test clips, extending training from 250 to 2,000 epochs raised top-1 by 7.9 points (two runs each; run_longtrain.sh, run_g rokking.sh); at 16 frames training length has saturated, a 4,000-epoch run gaining 0.6 points over 2,000.

5.5 Per-word behaviour

Twenty-seven of the 50 words reached at least 90% recall. The words with the lowest recall form visually similar groups, for example *big*/*wide*/*narrow* and *good*/*bad*/*wet*, which differ mainly in movement or in the relation between the two hands. The interface covers these cases through its Not-sure screen, which offers the five highest-ranked words; top-5 accuracy is 96.4%.

5.6 Cross-corpus test

The three released files were scored on CISLR without any retraining, with landmarks extracted by the same pipeline (report/final/scripts/run_crosscorpus_release.sh). On the 98 CISLR clips of deployed words, with the 50-word restriction, top-1 was 29.3% (29.6, 27.6, 30.6; chance 2%) and top-5 46.3%; answering from all 262 words on all 609 clips, top-1 was 9.6% (9.5, 8.7, 10.5) and top-5 22.4%. CISLR is recorded at 300 pixels by different signers, rooms and cameras, and MediaPipe finds a genuine left hand in 25.0% of its frames against 92.0% on the INCLUDE test clips (runs/gan_study/detection/cislr_bicubic.json), so the recogniser operates on far fewer detected hands there.

5.7 Super-resolution and the GAN

Every learned upscaler beat bicubic at every resolution; at 180p the gain was 14.3 to 15.0 points (SR-L1: +15.0, 269 clips gained and 57 lost, $p = 4.5 \times 10^{-34}$). The adversarial term added a further gain at 240p, where v2 over SR-L1 (+1.5 points, $p = 0.022$) and v3 over its control (+2.4, $p = 0.001$) both met the pre-registered rule; at 180p and 360p the GAN and no-GAN arms of each version were

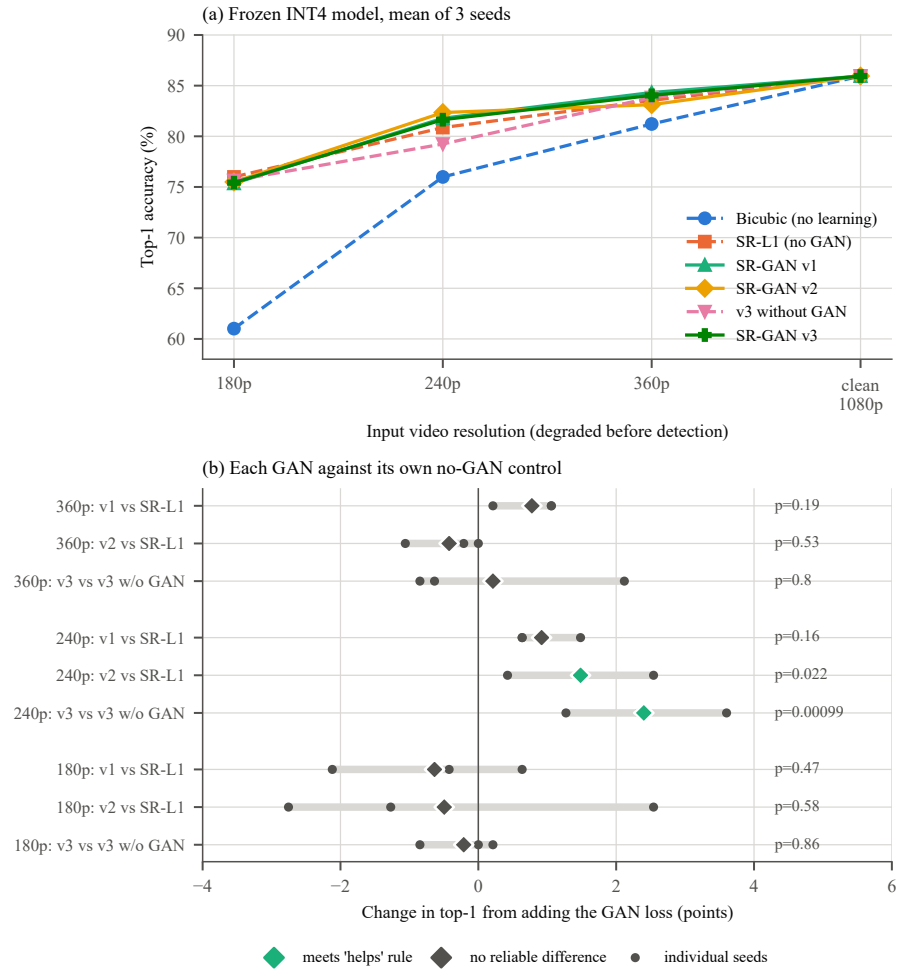


Fig. 11 Learned upscaling versus bicubic, and each GAN versus its own no-GAN control. (a) Mean top-1 of the three frozen released models on the 472 test clips after each front end; dashed lines are arms without an adversarial loss, and clean 1080p is the headline 85.9%. (b) Change in top-1 when the adversarial loss is added to an otherwise identical generator: diamonds are three-seed means, dots individual seeds, and p is the pooled exact McNemar test.

Data: runs/gan_study/frozen_int4/ and runs/gan_study_v1/frozen_int4/ (keys `masked_tta.top1`, `hits_masked_tta`); verdict rule gan_report.py:98, recomputed independently by audit_results.py with agreement on every cell.

Table 5 Super-resolution study at 180p (frozen INT4 models, 472 clips, three seeds). Detection is the share of frames with a genuine left-hand detection; on the clean test cache it is 92.0%. Image quality is measured on 1,200 held-out patches.

Front end	GAN loss	Top-1 (%)	Left hand found (%)	PSNR (dB)	SSIM
Bicubic	–	61.0	51.0	30.07	0.856
SR-L1	no	76.0	68.2	31.83	0.900
SR-GAN v1	yes	75.4	67.9	31.07	0.883
SR-GAN v2	yes	75.5	68.6	31.09	0.884
v3 without GAN	no	75.6	65.5	29.96	0.863
SR-GAN v3	yes	75.4	77.6	29.76	0.870

Sources: runs/gan_study/frozen_int4/L3_*.json and runs/gan_study_v1/frozen_int4/L3_*.json; runs/gan_study/detection/L3_*.json and runs/gan_study_v1/detection/L3_srgan.json (`streams.left_hand.genuine`); runs/gan_study/sr_quality*.json and runs/gan_study_v1/sr_quality.json (level `L3 180p`).

within about one point of each other, so learned upscaling accounts for nearly all of the recovery. SR-GAN v3 found the most hands at 180p (77.6% of frames against 68.2% for SR-L1). In line with the perception–distortion trade-off [32], the GANs produced sharper patches with lower PSNR than SR-L1.

After the classifiers were fine-tuned for 150 epochs on 360p video through each front end, the GAN front ends reached 77.8% (v1) and 76.9% (v2) at 180p against 64.4% for bicubic. A 2×2 decomposition at 180p with SR-GAN v2, crossing clean or degraded pixels with clean or degraded landmarks, attributed 7.7 of the 10.5 points between the 180p and the clean result to degraded landmarks and 2.9 to degraded pixels (runs/gan_study/diagnostics/), which locates the remaining headroom in hand-joint quality rather than in pixel texture.

Table 6 Integration and testing of each model variant through the application.

Model	Selected by	Tested through `POST /predict`	Benchmarked (472 clips)
Landmark QAT INT4 (released, seed 0)	default `--run`	50 clips: 43 correct, 48 in top five; 50 of 50 outcomes match offline	yes, 3 seeds
Landmark FP32 (seed 0)	--runruns/f16_262 w_lm_s0	50 clips: 43 correct; 50 of 50 match offline	yes, 3 seeds
Pixels-only FP32 (seed 0)	--runruns/f16_262 w_s0	50 clips: 35 correct; 50 of 50 match offline	yes, 3 seeds
Bicubic, SR-L1 and SR-GAN v2 front ends	`--sr` generator	50 clips at 180p, JPEG 40: bicubic 32, SR-L1 39, SR-GAN v2 37 correct; 50 of 50 match the study's seed-0 result in each arm	yes, 3 seeds

Each test posts raw video to the server started by `islvit.demo` and compares the top word with the same model's per-clip outcome in its `masked50_clean.json`. The 50 clips are the first test clip of each deployed word; 180p copies are stored losslessly (report/final/scripts/make_degraded_clips.py). Script report/final/scripts/app_integration_test.py; outputs report/final/data/app_integration_*.json.

5.8 Application integration and testing

Table 6 lists, for each model behind the application, how it is selected, how it was tested through the application, and its benchmark.

For each of the three classifiers, all 50 server answers had the same correctness as the offline evaluation of that model on that clip, including the clips the model gets wrong, and the same held for the three low-resolution front ends. The application therefore runs the evaluated computation: the same detector settings, 16 of 32 frames, six views and the 50-word restriction. The ordering of the front ends on these clips (SR-L1 39, SR-GAN v2 37, bicubic 32 correct) follows the full study. Screenshots of the seven screens are in report/review3/assets/; the threshold chart on the Settings screen plots the gating curve in runs/gate_mixed.json (three pixel-only models: 88.7% accuracy at 75.2% coverage for $\tau = 0.4$).

6 Discussion and originality

What moved accuracy.

The largest single gain came from an input the pipeline already computed: hand joints added 12.8 points for 91,008 parameters. Doubling the frames from 8 to 16 added about 10 points, and training longer helped until 16 frames, then saturated. With about 11 training clips per deployed word (571 clips for 50 words), explicit handshape measurements probably give the network information that 64-pixel crops carry only implicitly, which is consistent with larger crops adding tokens without adding accuracy.

What compression costs.

Group-wise 4-bit storage reduces the weights from about 15.6 MB to 2.07 MB. Rounding after training gave 83.4%; training with the rounding in the loop gave 85.9%, level with the FP32 model within seed noise. The release file is a storage format; the server rebuilds FP32 weights from it at load time.

What the protocol changes.

A 28-point gap between random-video and session-disjoint splits for one recipe suggests that INCLUDE accuracies reported on random splits largely measure recognition within recording sessions. Published INCLUDE-50 numbers [1, 4] are measured on the dataset's own split and are therefore a different quantity from the 85.9% measured here on held-out sessions.

What the GAN study shows.

Learned upscaling recovered most of the accuracy lost at 180p, and the adversarial term added a measurable gain at 240p. The 2×2 decomposition shows where the remaining headroom is: after upscaling, most of the gap to clean video lies in where the detector places hand joints. This motivated the v3 generator, trained against a hand-landmark loss, and it raised the share of detected hands at 180p from 68.2% to 77.6%.

Limitations.

All headline results come from one INCLUDE test set of 472 clips over 50 words, with three seeds per variant. Recording sessions were reconstructed from camera capture counters rather than taken from dataset metadata. Latency was measured on a desktop CPU, not the target wearable.

Originality.

The components of the system are established methods: factorised video Transformers, masked-image pretraining, MediaPipe landmarks, straight-through quantisation, group-wise scales and GAN super-resolution. The contribution is system-level and experimental. At the system level it is a region-crop factorised Transformer with additive position, detection-status and landmark conditioning, sized and quantised to a 2 MB file, and served by an application that runs the evaluated code path. At the experimental level it is a set of seed-replicated, controlled measurements of each design lever on one locked test set, a session-aware split with a measured leakage gap, a cross-corpus measurement on CISLR, and a GAN study with a no-GAN control. Table 7 ties each claim to its evidence.

Table 7 Claimed contributions, closest prior approach and evidence.

Contribution	Closest prior approach	Evidence in this project
Crops and landmarks in one token stream	pose-only models [4, 7]; multimodal ensembles [8]	+12.8 points over pixels only, 3 seeds, paired $p \leq 5 \times 10^{-9}$ per seed (Fig. 7)
2 MB 4-bit model	QAT [22]; group scales [25, 26]	2,069,700 B; PTQ 83.4% vs QAT 85.9%, pooled $p = 0.0012$; QAT level with FP32 (pooled $p = 0.44$)
Session-aware evaluation	user-independent benchmark split [3]	13 blocks, none crossing train/test; 98.5% vs 70.1% for one recipe
Evaluated code path in the app	–	300 server answers (three classifiers, three front ends), all matching offline outcomes (Table 6)
GAN with a no-GAN control	task-driven SR [34]; SR for vision tasks [33]	learned SR +14 to +15 points at 180p; adversarial term +1.5 to +2.4 at 240p

Among the works reviewed in Section 2, no ISL word recogniser reports these properties together.

7 Conclusion and future work

The project met its targets under the conditions it measured: 85.9% top-1 on 50 deployed words from held-out INCLUDE sessions, in a 2,069,700-byte 4-bit file, served by an application that reproduces the offline per-clip outcomes. Each design choice rests on controlled comparisons on one locked test set with three seeds. Three next steps extend the work: record the planned signer-disjoint set (three signers, 50 words, three takes; protocol in docs/CAPTURE_PROTOCOL.md) and score the released model on it unchanged; train a pose-only graph network on the same split as a second architecture family; and run the model with integer kernels on a phone-class processor, timing the detector and the classifier separately.

Acknowledgements. We carried out this work for the course 23CSE473 Neural Networks and Deep Learning at Amrita School of Computing, Amrita Vishwa Vidyapeetham, Coimbatore, supervised by Dr. T. Bagyammal. Figures elsewhere in the repository that show INCLUDE frames carry the dataset's CC-BY-4.0 attribution.

Declarations

Code availability. All code, configurations, split definitions, result files and the three released models are in the public repository https://github.com/not-good-keeper/compact-vit-sign-language. Links in this paper point to commit `3330b87d607eac34615517c092f79fc3e3abf1b6`. Files written for this paper are under `report/final/` (scripts, extracted data, figure sources and the evidence ledger).

Data availability. INCLUDE (Zenodo record 4010759), CISLR and iSign are available from their authors. The repository does not redistribute video.

Use of AI assistance. The audit scripts, figures and draft text of this paper were prepared with the assistance of an AI model and checked against the repository's result files.

Table A1 Experiment traceability. Links are pinned to commit `3330b87`.

ID	Question; controlled change	Launch	Raw result	Shown in
E1	Landmark gain; landmark stream on or off	run_landmarks.sh	f16_262w_s*/masked50_clean.json, f16_262w_lm_s*/masked50_clean.json	Tab. 3, Fig. 7
E2	4-bit cost; PTQ or QAT	run_lm_qat.sh	f16_262w_lm_s*/ptq4_clean.json, f16_262w_lm_s*_qat4/masked50_clean.json	Tab. 3
E3	Clean test cache; fresh detector per video	run_clean_test.sh	masked50_fresh.json vs `_clean`	Sec. 3.3
E4	Leakage size; split protocol	run_frames32.sh	baseline_random_s0, f16_clean_s0, f16_clean_s1	Fig. 8a
E5	Protocol ladder; split protocol	run_phase2.sh	full263_v2__random-video, full263_v2__take-group, sd_full263	Fig. 8b
E6	Checkpoint choice, folding; last vs best-val, validation folded	run_iter3.sh	sd263_lastsel, sd263_foldval	Sec. 3.6
E7	Seed noise; seed only (5 seeds)	run_lever2.sh	sd263_ssl_foldval, `_s1` to `_s4`	Sec. 3.11
E8	Temporal length; frames drawn	run_frames32.sh	f8_clean_s0, f24_clean_s0, f32_clean_s0	Fig. 9a
E9	Crop size; input pixels	run_ablation_res.sh	abl64__take-group, abl112__take-group, abl128__take-group	Fig. 9b
E10	Training length; epochs	run_longtrain.sh, run_push75.sh	long2000_s1, grok_wd005, f16_long4k_s0	Sec. 5.4
E11	Super-resolution; front end	run_gan_study.sh, run_gan_v3_test.sh	runs/gan_study/, runs/gan_study_v1/	Fig. 11, Tab. 5
E12	App route; model behind the server	app_integration_test.py*	report/final/data/app_integration_*.json*	Tab. 6
E13	Released model on CISLR; corpus	run_crosscorpus_release.sh*	report/final/data/crosscorpus_release_s*.json*	Sec. 5.6

*New in this report (`report/final/`). Evaluator for E1 to E3 and E11: mask50.py; for E4 to E10: train.py, which scores the test split at the end of training.

Appendix A Experiment traceability

Appendix B Architecture design and model iterations

Every number below comes from the main text or from the run file linked beside it; this appendix adds no new measurement.

B.1 What the architecture borrows

ISL-ViT-Tiny is assembled from four published designs; Table B2 lists what each contributes and what was changed. The largest borrowing is the *factorised encoder* of ViViT [5]. ViViT describes four ways to apply a Transformer to video. Its Model 1 attends jointly over every space–time token; Model 2, the factorised encoder, runs a spatial Transformer on the tokens of each frame, reads out one [CLS] embedding per frame, and passes the sequence of frame embeddings to a temporal Transformer; Models 3 and 4 factorise attention inside each block instead. TimeSformer's divided space–time attention [6] works like Model 3, alternating temporal and spatial attention within every block. ISL-ViT-Tiny follows Model 2 because no attention layer in it ever spans all the patches of a clip: the spatial stage attends within one image, and the temporal stage sees one token per image.

The change from ViViT is the unit that the spatial stage encodes. ViViT encodes a whole frame; ISL-ViT-Tiny encodes a detected crop, three per frame, so Stage B receives $T \cdot S = 48$ tokens rather than $T = 16$ and needs a stream embedding to tell the left hand, the right hand and the face apart. Stage B therefore attends across time and across body parts in one sequence, so it can relate the two hands to each other and to the face at any pair of frames. The attention cost of this arrangement can be counted directly. Per head and per layer, Stage A computes $48 \times 17^2 = 13{,}872$ attention scores and Stage B $49^2 = 2{,}401$, a total of 16,273. Joint attention over the 816 patch tokens of the clip would compute $816^2 = 665{,}856$, about 41 times as many.

Inside each stage the building blocks are those of ViT [16]: a strided convolution as patch embedding, a learned [CLS] token, a learned position embedding and pre-norm Transformer blocks. The dimensions are those of DeiT-Tiny [17] (width 192, three heads, MLP width 768), chosen so that ImageNet weights load into Stage A without a projection layer. The loader copies the patch embedding, the [CLS] token and the first four of the twelve pretrained blocks, and resizes the 14×14 position grid to the 4×4 grid of a 64-pixel crop by bicubic interpolation (isl_vit.py:347). The same width is what the iSign self-supervised checkpoint was trained at, which is why the model was never made narrower.

The self-supervised initialisation follows SimMIM [39] rather than the masked autoencoder (MAE) of He et

Table B2 Components taken from earlier Vision Transformers and what ISL-ViT-Tiny changes.

Component	Source	Kept	Changed
Factorised encoder	ViViT Model 2 [5]	spatial Transformer per image, [CLS] readout, temporal Transformer over the readouts	the image is a hand or face crop, not a frame; 48 tokens per clip with stream embeddings
Patch embedding, [CLS], position embedding, pre-norm block	ViT [16]	all, unchanged	64-pixel input gives a 4×4 grid of 16 patches
Width 192, 3 heads, MLP 768	DeiT-Tiny [17]	dimensions and the first four pretrained blocks	four blocks per stage instead of twelve
Masked image modelling	SimMIM [39]	learned mask token inside the encoder, pixel regression on masked patches	mask drawn across time, stream and space; crops without a real detection excluded from the loss
Stochastic depth	Huang et al. [38]	drop rate rising to 0.1 with depth	–
Token conditioning	this work	–	crop geometry, missing-detection flag, hand-landmark and body-pose vectors added to the crop tokens through zero-initialised MLPs

al. [48]. The mask hides 75% of the patches and is drawn jointly over time, stream and space, because a mask drawn inside each crop would let Stage A solve the task alone and leave Stage B untrained. MAE removes the masked tokens and encodes only the visible ones. With 16 patches per crop and a mask drawn over the whole clip, that would leave about four visible patches per crop, a different number in each crop. SimMIM instead replaces each masked patch with a learned mask token inside the encoder, so every crop keeps 17 tokens and the encoder that is pretrained is exactly the encoder that is later fine-tuned.

B.2 What is added, and why

Cropping removes two things a sign depends on: where the hand is, and its exact shape, which 64 pixels resolve only coarsely. The additions put both back without adding a single token. The box centre and size, a flag for crops that were interpolated rather than detected, a 65-number handshape and wrist-position vector per hand and a 21-number body-pose vector are each mapped to width 192 and added to the crop token they describe (Eq. (3)). The attention pattern, the sequence length and the temporal stage are therefore the same as in the pixel-only model. The last layer of every added MLP starts at zero, so at the first training step the network computes exactly what the pretrained pixel-only network computes. The side inputs then enter gradually, which is intended to keep early gradients from disturbing the transferred weights. The landmark features are computed inside the model from raw joint coordinates, so training, offline evaluation and the application cannot compute them differently.

B.3 How the model reaches 2 MB

The architecture fixes the parameter count at 3,894,214; quantisation sets the bytes. Stored in FP32, the weights take $3{,}894{,}214 \times 4 = 15{,}576{,}856$ bytes, about 15.6 MB. For the 3.76 M-parameter pixel-only model, 8-bit weights still took 3.70 MB, nearly twice the budget (export.py:246). Making the network smaller was the other route, and it was costly in a specific way: the width cannot fall below 192 without losing both the ImageNet and the self-supervised initialisations. A distillation student with two blocks per stage at width 192 was built for this purpose and measures 1.95 MB packed (distill.py). No training run of it is recorded: 4-bit weights met the budget with the full model first.

The 4-bit format stores every weight matrix in groups of 128 consecutive weights (Eq. (4)). Each group keeps one FP16 scale, $\sigma_G = \max_{w \in G} |w|/7$, and 128 signed codes in $[-8, 7]$, packed two per byte. One group therefore costs $128 \times 4 + 16 = 528$ bits, or 4.125 bits per weight, and the scales add 3.1% to the codes. A single scale per matrix would be cheaper still, but the largest weight in a 768×192 matrix would then set the step size for all 147,456 weights and leave most of them on a few levels; a group of 128 confines each outlier's effect to its own group. Embeddings, LayerNorm parameters and biases stay in FP16. At 4.125 bits per weight the 3.89 M parameters would need about 2.0 MB. The released file, 2,069,700 bytes, also holds the FP16 tensors, the configuration and the word list. Relative to FP32 the file is 7.5 times smaller.

Rounding a trained FP32 model this way lowered masked-50 accuracy from 86.5% to 83.4% (ptq4_clean.json). Quantisation-aware training recovers most of the loss. For 300 further epochs the forward pass uses the rounded weights and the gradient updates full-precision master weights through a straight-through estimator [22, 23]; the released model scores 85.9%, within seed noise of FP32. The scale is rounded to FP16 before the weights are reconstructed, and training and export call the same rounding functions, so the model that is scored is the file that is shipped (export.py:202).

Table B3 Model iterations in the order they were made. Each step keeps everything above it in the same block. Values are means over the seeds shown.

Step	Change	Top-1 (%)	Seeds	Run
262 words, 1,010 test clips, single view				
1	Factorised crop ViT, 8 frames, ImageNet initialisation, best-validation checkpoint	22.1	1	sd_full263
2	Final weights kept; validation clips folded into training (1,957 to 2,845 clips)	33.0	1	sd263_foldval
3	Masked image modelling on CISLR; resolution jitter during fine-tuning	39.9	2	sd263_ssl_rj_s*
4	Masked image modelling on 17,896 iSign clips instead	42.1	2	sd263_isign_rj_s*
50 words, 472 test clips, single view				
5	Step 4 recipe on the 50-word tier, 250 epochs	51.7	2	vocab50_s*
6	2,000 epochs	59.6	2	long2000_s1, grok_wd005
7	16 frames instead of 8	66.1	2	cap_16f64, cap_16f64_s1
8	16 frames drawn from a 32-frame cache; CISLR clips removed from training	70.1	2	f16_clean_s*
262 words trained, restricted to 50 at test, 472 test clips, six-view TTA				
9	Train on all 262 words, 1,000 epochs	73.7	3	f16_262w_s*
10	Hand-landmark and body-pose inputs	86.5	3	f16_262w_lm_s*
11	4-bit, rounded after training	83.4	3	ptq4_clean.json
12	4-bit, quantisation-aware training (released, 2,069,700 bytes)	85.9	3	f16_262w_lm_s*_qat4

B.4 Model iterations

The FP32 model reached 86.5% only after more than 80 recorded training and pretraining runs, spread over successive iterations of the network, its inputs and its training recipe, and only then was the landmark model quantised. Table B3 lists the steps that changed accuracy, in the order they were made. Three test sets were used along the way, so a comparison is valid only within a block of the table. Steps 1 to 4 were scored on all 262 words of the 1,010 held-out-session INCLUDE clips; steps 5 to 8 trained on the 50 deployed words only and were scored on the 472 test clips; steps 9 to 12 are the final protocol of Section 3.11.

Apart from the landmark inputs, the steps that mattered most were not architectural. Fixing checkpoint selection and folding validation into training (step 2) and training for longer (step 6) changed the recipe, not the network. Self-supervised initialisation helped once the input blur of the pretraining corpus was matched by resolution jitter (step 3). Doubling the frames from 8 to 16 (step 7) was the largest change to the model's input until landmarks. Step 8 changed two things at once, the 32-frame cache and the removal of CISLR clips from training, so its 4-point gain cannot be split between them; a cache with spare frames probably helps because the sampler can then jitter within each segment. Training on all 262 words and restricting the output to 50 at test time (step 9) scored 73.7% against 72.7% for the 50-word models of step 8 under the same six-view protocol, a difference within seed noise, and it keeps the other 212 words available. The hand landmarks (step 10) added 12.8 points for 91,008 parameters, more than every earlier change to resolution, capacity or schedule.

Several candidates were measured and dropped. A 96-pixel input lowered 16-frame accuracy from 66.1% to 61.0% (cap_16f96, one seed). Pretraining natively on 16-frame clips reached a lower reconstruction loss but transferred worse, 60.0% against 70.1% (f16_mim16_s0). Fine-tuning the 262-word model on the 50-word softmax moved two seeds significantly in opposite directions (f16_262w_s*_narrow). Five further landmark variants, screened on validation clips with the test set untouched, gave no gain that held across seeds. The first two of these decisions rest on one seed each.

References

[1] Sridhar, A., Ganesan, R.G., Kumar, P., Khapra, M.: INCLUDE: A large scale dataset for Indian sign language recognition. In: Proceedings of the 28th ACM International Conference on Multimedia, pp. 1366–1375 (2020). https://doi.org/10.1145/3394171.3413528

[2] Kapoor, S., Narayanan, A.: Leakage and the reproducibility crisis in machine-learning-based science. Patterns **4**(9), 100804 (2023) https://doi.org/10.1016/j.patter.2023.100804

[3] Sincan, O.M., Keles, H.Y.: AUTSL: A large scale multi-modal Turkish sign language dataset and baseline methods. IEEE Access **8**, 181340–181355 (2020) https://doi.org/10.1109/ACCESS.2020.3028072

[4] Selvaraj, P., NC, G., Kumar, P., Khapra, M.: OpenHands: Making sign language recognition accessible with pose-based pretrained models across languages. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 2114–2133 (2022). https://doi.org/10.18653/v1/2022.acl-long.150

[5] Arnab, A., Dehghani, M., Heigold, G., Sun, C., Lučić, M., Schmid, C.: ViViT: A video vision transformer. In: 2021 IEEE/CVF International Conference on Computer Vision (ICCV), pp. 6836–6846 (2021).

https://doi.org/10.1109/ICCV48922.2021.00676

[6] Bertasius, G., Wang, H., Torresani, L.: Is space-time attention all you need for video understanding? In: Proceedings of the 38th International Conference on Machine Learning. PMLR, vol. 139, pp. 813–824 (2021). https://arxiv.org/abs/2102.05095

[7] Boháček, M., Hrúz, M.: Sign pose-based transformer for word-level sign language recognition. In: 2022 IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW), pp. 182–191 (2022). https://doi.org/10.1109/WACVW54805.2022.00024

[8] Jiang, S., Sun, B., Wang, L., Bai, Y., Li, K., Fu, Y.: Skeleton aware multi-modal sign language recognition. In: 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), pp. 3413–3423 (2021). https://doi.org/10.1109/CVPRW53098.2021.00380

[9] Lamaakal, I., Yahyati, C., Maleh, Y., El Makkaoui, K., Ouahbi, I.: An explainable hybrid CNN–transformer model for sign language recognition on edge devices using adaptive fusion and knowledge distillation. Scientific Reports **16**, 7143 (2026) https://doi.org/10.1038/s41598-026-38478-8

[10] Venkatappa, U., K S, N., K S, N., Bhat, P.P., Srinivas, P.: Dynamic Kannada sign language recognition on resource constrained devices. Scientific Reports **16**, 11186 (2026) https://doi.org/10.1038/s41598-026-40181-7

[11] Joshi, A., Bhat, A., S, P., Gole, P., Gupta, S., Agarwal, S., Modi, A.: CISLR: Corpus for Indian sign language recognition. In: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, pp. 10357–10366 (2022). https://doi.org/10.18653/v1/2022.emnlp-main.707

[12] Joshi, A., Mohanty, R., Kanakanti, M., Mangla, A., Choudhary, S., Barbate, M., Modi, A.: iSign: A benchmark for Indian sign language processing. In: Findings of the Association for Computational Linguistics: ACL 2024, pp. 10827–10844 (2024). https://doi.org/10.18653/v1/2024.findings-acl.643

[13] Nedungadi, P., Dileep, G., Geetha, M., Raman, R.: Vision transformer-powered conversational agent for real-time Indian sign language e-governance accessibility. Scientific Reports **15**, 33055 (2025) https://doi.org/10.1038/s41598-025-12667-3

[14] Kaliyaperumal, V.G., Gopalan, P.A.: A deep neural network framework for dynamic two-handed Indian sign language recognition in hearing and speech-impaired communities. Sensors **25**(12), 3652 (2025) https://doi.org/10.3390/s25123652

[15] Damdoo, R., Kumar, P.: An integrative survey on Indian sign language recognition and translation. IET Image Processing **19**(1), e70000 (2025) https://doi.org/10.1049/ipr2.70000

[16] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N.: An image is worth 16x16 words: Transformers for image recognition at scale. In: International Conference on Learning Representations (ICLR) (2021). https://arxiv.org/abs/2010.11929

[17] Touvron, H., Cord, M., Douze, M., Massa, F., Sablayrolles, A., Jégou, H.: Training data-efficient image transformers & distillation through attention. In: Proceedings of the 38th International Conference on Machine Learning. PMLR, vol. 139, pp. 10347–10357 (2021). https://arxiv.org/abs/2012.12877

[18] Camgoz, N.C., Koller, O., Hadfield, S., Bowden, R.: Sign language transformers: Joint end-to-end sign language recognition and translation. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 10023–10033 (2020). https://doi.org/10.1109/CVPR42600.2020.01004

[19] Hu, H., Zhao, W., Zhou, W., Wang, Y., Li, H.: SignBERT: Pre-training of hand-model-aware representation for sign language recognition. In: 2021 IEEE/CVF International Conference on Computer Vision (ICCV), pp. 11087–11096 (2021). https://doi.org/10.1109/ICCV48922.2021.01090

[20] Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M.G., Lee, J., Chang, W.-T., Hua, W., Georg, M., Grundmann, M.: MediaPipe: A Framework for Building Perception Pipelines (2019). https://arxiv.org/abs/1906.08172

[21] Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., Grundmann, M.: MediaPipe Hands: On-device Real-time Hand Tracking. CVPR Workshop on Computer Vision for Augmented and Virtual Reality (2020). https://arxiv.org/abs/2006.10214

[22] Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., Kalenichenko, D.: Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 2704–2713 (2018). https://doi.org/10.1109/CVPR.2018.00286

[23] Bengio, Y., Léonard, N., Courville, A.: Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation (2013). https://arxiv.org/abs/1308.3432

[24] Nagel, M., Fournarakis, M., Amjad, R.A., Bondarenko, Y., Baalen, M., Blankevoort, T.: A White

Paper on Neural Network Quantization (2021). https://arxiv.org/abs/2106.08295

[25] Shen, S., Dong, Z., Ye, J., Ma, L., Yao, Z., Gholami, A., Mahoney, M.W., Keutzer, K.: Q-BERT: Hessian based ultra low precision quantization of BERT. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 34, pp. 8815–8821 (2020). https://doi.org/10.1609/aaai.v34i05.6409

[26] Frantar, E., Ashkboos, S., Hoefler, T., Alistarh, D.: GPTQ: Accurate post-training quantization for generative pre-trained transformers. In: International Conference on Learning Representations (ICLR) (2023). https://arxiv.org/abs/2210.17323

[27] Li, D., Rodriguez Opazo, C., Yu, X., Li, H.: Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison. In: 2020 IEEE Winter Conference on Applications of Computer Vision (WACV), pp. 1459–1469 (2020). https://doi.org/10.1109/WACV45572.2020.9093512

[28] Ledig, C., Theis, L., Huszár, F., Caballero, J., Cunningham, A., Acosta, A., Aitken, A., Tejani, A., Totz, J., Wang, Z., Shi, W.: Photo-realistic single image super-resolution using a generative adversarial network. In: 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4681–4690 (2017). https://doi.org/10.1109/CVPR.2017.19

[29] Wang, X., Yu, K., Wu, S., Gu, J., Liu, Y., Dong, C., Qiao, Y., Loy, C.C.: ESRGAN: Enhanced super-resolution generative adversarial networks. In: Computer Vision – ECCV 2018 Workshops. Lecture Notes in Computer Science, pp. 63–79 (2019). https://doi.org/10.1007/978-3-030-11021-5_5

[30] Wang, X., Xie, L., Dong, C., Shan, Y.: Real-ESRGAN: Training real-world blind super-resolution with pure synthetic data. In: 2021 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW), pp. 1905–1914 (2021). https://doi.org/10.1109/ICCVW54120.2021.00217

[31] Goodfellow, I.J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y.: Generative adversarial nets. In: Advances in Neural Information Processing Systems 27 (2014). https://arxiv.org/abs/1406.2661

[32] Blau, Y., Michaeli, T.: The perception-distortion tradeoff. In: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 6228–6237 (2018). https://doi.org/10.1109/CVPR.2018.00652

[33] Dai, D., Wang, Y., Chen, Y., Van Gool, L.: Is image super-resolution helpful for other vision tasks? In: 2016 IEEE Winter Conference on Applications of Computer Vision (WACV), pp. 1–9 (2016). https://doi.org/10.1109/WACV.2016.7477613

[34] Haris, M., Shakhnarovich, G., Ukita, N.: Task-Driven Super Resolution: Object Detection in Low-resolution Images (2018). https://arxiv.org/abs/1803.11316

[35] Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. In: International Conference on Learning Representations (ICLR) (2019). https://arxiv.org/abs/1711.05101

[36] Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., Wojna, Z.: Rethinking the inception architecture for computer vision. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2818–2826 (2016). https://doi.org/10.1109/CVPR.2016.308

[37] Zhang, H., Cisse, M., Dauphin, Y.N., Lopez-Paz, D.: mixup: Beyond empirical risk minimization. In: International Conference on Learning Representations (ICLR) (2018). https://arxiv.org/abs/1710.09412

[38] Huang, G., Sun, Y., Liu, Z., Sedra, D., Weinberger, K.Q.: Deep networks with stochastic depth. In: Computer Vision – ECCV 2016. Lecture Notes in Computer Science, pp. 646–661 (2016). https://doi.org/10.1007/978-3-319-46493-0_39

[39] Xie, Z., Zhang, Z., Cao, Y., Lin, Y., Bao, J., Yao, Z., Dai, Q., Hu, H.: SimMIM: a simple framework for masked image modeling. In: 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 9653–9663 (2022). https://doi.org/10.1109/CVPR52688.2022.00943

[40] Johnson, J., Alahi, A., Fei-Fei, L.: Perceptual losses for real-time style transfer and super-resolution. In: Computer Vision – ECCV 2016. Lecture Notes in Computer Science, pp. 694–711 (2016). https://doi.org/10.1007/978-3-319-46475-6_43

[41] Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition. In: International Conference on Learning Representations (ICLR) (2015). https://arxiv.org/abs/1409.1556

[42] Schönfeld, E., Schiele, B., Khoreva, A.: A U-Net based discriminator for generative adversarial networks. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 8207–8216 (2020). https://doi.org/10.1109/CVPR42600.2020.00823

[43] Miyato, T., Kataoka, T., Koyama, M., Yoshida, Y.: Spectral normalization for generative adversarial networks. In: International Conference on Learning Representations (ICLR) (2018). https://arxiv.org/abs/1802.05957

[44] Jolicoeur-Martineau, A.: The relativistic discriminator: a key element missing from standard GAN. In: International Conference on Learning Representations (ICLR) (2019). https://arxiv.org/abs/1807.00734

[45] He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778 (2016). https://doi.org/10.1109/CVPR.2016.90

[46] Sun, X., Xiao, B., Wei, F., Liang, S., Wei, Y.: Integral human pose regression. In: Computer Vision – ECCV 2018. Lecture Notes in Computer Science, pp. 536–553 (2018). https://doi.org/10.1007/978-3-030-01231-1_33

[47] McNemar, Q.: Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika **12**(2), 153–157 (1947) https://doi.org/10.1007/BF02295996

[48] He, K., Chen, X., Xie, S., Li, Y., Dollár, P., Girshick, R.: Masked autoencoders are scalable vision learners. In: 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 16000–16009 (2022). https://arxiv.org/abs/2111.06377